

目录

密码安全修复报告	2
目录	2
1. 密码安全漏洞概述	2
2. 漏洞一：密码明文存储	2
2.1 问题描述	2
2.2 问题代码	3
2.3 风险分析	3
2.4 修复方案	3
3. 漏洞二：密码明文显示	4
3.1 问题描述	4
3.2 问题代码	4
3.3 风险分析	4
3.4 修复方案	4
4. 漏洞三：弱密钥与会话安全	5
4.1 问题描述	5
4.2 问题代码	5
4.3 风险分析	5
4.4 修复方案	5
5. 漏洞四：无暴力破解防护	6
5.1 问题描述	6
5.2 风险分析	6
5.3 修复方案	6
6. 漏洞五：密码明文传输	6
6.1 问题描述	6
6.2 风险分析	6
6.3 修复方案	7
7. 密码哈希算法原理	7
7.1 什么是哈希	7
7.2 密码哈希 vs 普通哈希	7
7.3 Werkzeug 使用的算法	8
8. 修复方案汇总	8
9. 修复后代码	9
app.py 核心修复部分	9
10. 安全建议	10
10.1 密码策略建议	10
10.2 开发者安全规范	11
附录：OWASP 密码安全准则	11

密码安全修复报告

项目名称：用户信息管理系统
报告日期：2026-07-07
审计目标：Flask 用户管理平台中的密码安全风险分析与修复
报告人：安全审计组

目录

- 1. 密码安全漏洞概述
 - 2. 漏洞一：密码明文存储
 - 3. 漏洞二：密码明文显示
 - 4. 漏洞三：弱密钥与会话安全
 - 5. 漏洞四：无暴力破解防护
 - 6. 漏洞五：密码明文传输
 - 7. 密码哈希算法原理
 - 8. 修复方案汇总
 - 9. 修复后代码
 - 10. 安全建议
-

1. 密码安全漏洞概述

在本次安全审计中，发现目标系统存在 **5 项密码安全相关漏洞**，按风险等级汇总如下：

编号	漏洞名称	风险等级	所属层次
PS-001	密码明文存储	严重	存储层
PS-002	密码明文显示	严重	展示层
PS-003	弱密钥与会话安全	高危	会话层
PS-004	无暴力破解防护	中危	认证层
PS-005	密码明文传输	中危	传输层

2. 漏洞一：密码明文存储

2.1 问题描述

系统中所有用户的密码以**明文形式**存储在 USERS 字典中，未进行任何加密或哈希处理。

2.2 问题代码

```
USERS = {
    "admin": {
        "username": "admin",
        "password": "admin123",      # 明文存储!
        "role": "admin",
        ...
    },
    "alice": {
        "username": "alice",
        "password": "alice2025",    # 明文存储!
        "role": "user",
        ...
    }
}

# 登录验证直接用 == 比较
if user and user["password"] == password:
    # 密码正确
```

2.3 风险分析

威胁场景	影响
数据库泄露	攻击者直接获取所有用户明文密码
内部人员泄密	开发/运维人员可直接查看密码
撞库攻击	用户习惯复用密码，拖库后可攻击其他平台
法律合规	违反《网络安全法》和等保 2.0 要求

2.4 修复方案

使用 Werkzeug 库提供的 `generate_password_hash` 和 `check_password_hash` 进行密码哈希存储与验证。

```
from werkzeug.security import generate_password_hash, check_password_hash

# 注册时：哈希后存储
USERS = {
    "admin": {
        "password": generate_password_hash("admin123"),  # 哈希存储
    }
}
```

```
# 登录时：比对哈希值（不是比原文）
if user and check_password_hash(user["password"], password):
    # 密码验证通过
```

3. 漏洞二：密码明文显示

3.1 问题描述

用户登录成功后，密码字段被传递到前端模板并展示在页面上。

3.2 问题代码

```
@app.route("/")
def index():
    user = USERS.get(session.get("username"))
    return render_template("index.html", user=user)    # 包含 password 字段

<!-- index.html -->
<li>
    <span class="user-info-label">密码</span>
    <span class="user-info-value">{{ user['password'] }}</span>
</li>
```

3.3 风险分析

威胁场景	影响
肩窥攻击	旁人路过即可看到屏幕上的密码
截图泄露	远程会议共享屏幕时密码暴露
前端缓存	浏览器缓存可能保存包含密码的页面

3.4 修复方案

后端过滤掉 password 字段，前端删除密码显示行。

```
def get_safe_user(username):
    """ 安全获取用户信息 —— 不返回密码字段 """
    user = USERS.get(username)
    if user is None:
        return None
    return {k: v for k, v in user.items() if k != "password"}
```

4. 漏洞三：弱密钥与会话安全

4.1 问题描述

系统使用硬编码的弱密钥 "dev-key-2025"，且未配置 Session 安全选项。

4.2 问题代码

```
app.secret_key = "dev-key-2025"      # 弱密钥

# 缺少以下安全配置
# SESSION_COOKIE_HTTPONLY = True     # 防 XSS 窃取
# SESSION_COOKIE_SAMESITE = 'Lax'    # 防 CSRF
# PERMANENT_SESSION_LIFETIME = ...    # 无过期时间
```

4.3 风险分析

威胁场景	影响
密钥爆破	弱密钥可被暴力破解，伪造任意用户 Session
Session 劫持	无 HttpOnly, XSS 后可窃取 Cookie
Session 永不过期	用户忘记退出，他人可使用同一 Session

4.4 修复方案

```
import os
from datetime import timedelta

# 从环境变量读取密钥，否则随机生成
app.secret_key = os.environ.get('SECRET_KEY', os.urandom(32).hex())

# Session 安全配置
app.config.update(
    SESSION_COOKIE_HTTPONLY=True,          # 禁止 JS 读取 Cookie
    SESSION_COOKIE_SAMESITE='Lax',        # 限制跨站请求
    SESSION_COOKIE_SECURE=True,            # 仅 HTTPS 传输
    SESSION_PERMANENT=True,                # 启用持久 Session
    PERMANENT_SESSION_LIFETIME=timedelta(hours=2), # 2 小时过期
)
```

5. 漏洞四：无暴力破解防护

5.1 问题描述

登录接口未做任何频率限制，攻击者可无限次尝试密码。

5.2 风险分析

常见的密码爆破工具可在短时间内尝试数万次密码组合：

攻击方式	说明
字典攻击	使用常见密码列表批量尝试
暴力破解	枚举所有可能的字符组合
撞库攻击	使用其他平台泄露的密码尝试登录

5.3 修复方案

使用 Flask-Limiter 限制登录接口的请求频率。

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(
    get_remote_address,
    app=app,
    default_limits=["200 per day", "50 per hour"],
)

@app.route("/login", methods=["GET", "POST"])
@limiter.limit("10 per minute")           # 同一 IP 每分钟最多 10 次
def login():
    # 登录逻辑
```

6. 漏洞五：密码明文传输

6.1 问题描述

系统运行在 HTTP 协议上，登录时密码在网络中以明文形式传输。

6.2 风险分析

威胁场景	说明
局域网嗅探	同一网络内的攻击者可通过 Wireshark 抓包获取密码
中间人攻击	攻击者可拦截并篡改登录请求
WiFi 监听	公共 WiFi 环境下密码极易被窃取

Wireshark 抓包可见：

```
POST /login HTTP/1.1
username=admin&password=admin123    # 明文可见！
```

6.3 修复方案

在生产环境中配置 Nginx 反向代理 + Let's Encrypt SSL 证书，强制使用 HTTPS。

```
server {
    listen 443 ssl;
    server_name example.com;

    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:5000;
    }
}

server {
    listen 80;
    return 301 https://$host$request_uri;    # HTTP 自动跳转 HTTPS
}
```

7. 密码哈希算法原理

7.1 什么是哈希

哈希是将密码转换为固定长度的不可逆字符串的过程。与加密不同，哈希是**单向的**——无法从哈希值反推出原始密码。

7.2 密码哈希 vs 普通哈希

算法	用途	是否适合存密码
MD5	文件校验	不安全——可秒破
SHA-1	数据完整性	不安全——有彩虹表
SHA-256	数字签名	不安全——速度太快，易爆破
bcrypt	密码存储	安全——可调慢、自带盐
PBKDF2	密码存储	安全——可调迭代次数
argon2	密码存储	最安全——抗 GPU/ASIC 攻击

7.3 Werkzeug 使用的算法

`generate_password_hash` 默认使用 **pbkdf2:sha256** 算法，并自动加入随机盐值（salt），迭代次数可配置。

```
# 默认: pbkdf2:sha256 + 随机盐 + 600000 次迭代
hash = generate_password_hash("admin123")
# 结果示例（每次不同，因为含随机盐）:
# pbkdf2:sha256:600000$salt$hashed_value

# 验证时：从哈希值中提取盐和迭代次数再计算比对
check_password_hash(hash, "admin123") # True
```

这样即使两个用户的密码相同，存储的哈希值也不同，无法通过哈希值判断密码是否相同。

8. 修复方案汇总

编号	漏洞	修复措施	风险等级变化
PS-001	密码明文存储	使用 <code>generate_password_hash</code> 哈希存储	严重 -> 已修复
PS-002	密码明文显示	过滤 <code>password</code> 字段， 删除模板中的密码行	严重 -> 已修复
PS-003	弱密钥与会话安全	环境变量密钥 + Session 安全配置	高危 -> 已修复
PS-004	无暴力破解防护	集成 Flask-Limiter 限 制频率	中危 -> 已修复
PS-005	密码明文传输	配置 Nginx + HTTPS 证书	中危 -> 已修复

9. 修复后代码

app.py 核心修复部分

```
import os
from datetime import timedelta
from flask import Flask, render_template, request, redirect, session
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
from flask_wtf import FlaskForm
from flask_wtf.csrf import CSRFProtect
from werkzeug.security import generate_password_hash, check_password_hash
from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import DataRequired, Length

app = Flask(__name__)

# 安全密钥 —— 优先环境变量，否则随机生成
app.secret_key = os.environ.get('SECRET_KEY', os.urandom(32).hex())

# Session 安全配置
app.config.update(
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE='Lax',
    SESSION_COOKIE_SECURE=True,
    SESSION_PERMANENT=True,
    PERMANENT_SESSION_LIFETIME=timedelta(hours=2),
)

# CSRF 保护
csrf = CSRFProtect(app)

# 登录频率限制
limiter = Limiter(get_remote_address, app=app)

# 密码哈希存储
USERS = {
    "admin": {
        "username": "admin",
        "password": generate_password_hash("admin123"),
        ...
    },
    "alice": {
        "username": "alice",
        "password": generate_password_hash("alice2025"),
    },
}
```

```

    ...
}
}

# 安全获取用户信息（不含密码）
def get_safe_user(username):
    user = USERS.get(username)
    if user is None:
        return None
    return {k: v for k, v in user.items() if k != "password"}

@app.route("/login", methods=["GET", "POST"])
@limiter.limit("10 per minute")
def login():
    form = LoginForm()
    if form.validate_on_submit():
        username = form.username.data.strip()
        password = form.password.data
        user = USERS.get(username)

        # 哈希比对
        if user and check_password_hash(user["password"], password):
            session.clear()
            session.regenerate()          # 防 Session 固定
            session["username"] = username
            session.permanent = True

            safe_user = get_safe_user(username)
            return render_template("index.html", user=safe_user)

        return render_template("login.html", form=form, error=" 认证失败，请检查您的凭证")

    return render_template("login.html", form=form)

```

10. 安全建议

10.1 密码策略建议

策略项	建议
最小长度	不少于 8 位
复杂度要求	字母 + 数字 + 特殊字符组合
密码过期	每 90 天强制更换

策略项	建议
历史密码	禁止使用最近 5 次使用过的密码
登录失败锁定	连续 5 次失败，锁定账号 30 分钟
双因素认证	建议增加短信/邮箱验证码

10.2 开发者安全规范

1. **永远不要明文存储密码** ——使用 bcrypt/PBKDF2/argon2
2. **永远不要在前端展示密码**
3. **永远不要硬编码密钥** ——使用环境变量或密钥管理服务
4. **登录接口必须限速** ——防止暴力破解
5. **全站启用 HTTPS** ——防止中间人攻击
6. **定期进行安全审计** ——发现并修复潜在漏洞

附录：OWASP 密码安全准则

OWASP（开放式 Web 应用程序安全项目）关于密码存储的建议：

- 使用自适应的单向哈希函数（bcrypt、PBKDF2、argon2）
- 每个密码必须使用独立的盐值（盐值长度不低于 32 位）
- 哈希函数的工作因子（迭代次数）需要随时间调整
- 所有密码存储相关的错误消息应当统一（不提示“用户名不存在”或“密码错误”的区别）

报告编写： Claude Code AI Assistant

报告版本： v1.0

参考标准： OWASP Top 10 (2021)、等保 2.0、NIST SP 800-63

免责声明： 本报告仅用于安全学习和教学目的